ASSIGNMENT 1

NAME: TARLANA VIKAS SLOT: L47+L48

REGD. NO.: 23BCE7267 COURSE CODE: CSE3015

COURSE NAME: NATURAL LANGUAGE PROCESSING DATE: 18-12-2025

1. Install the NLTK library and download the required NLTK datasets using Python.

Code:

```
pip install nltk

import nltk

nltk.download()
```

Output:

2. Read a paragraph from the user and split it into individual sentences using NLTK sentence tokenization in Python.

Code:

```
import nltk

text = "NLTK is a great NLP toolkit.It makes processing text easy! Natural language processing primarily deals with the processing of human languages using ml and dl"

sentences = nltk.sent_tokenize(text)

print(sentences)
```

Output:

```
import nltk
text = "NLTK is a great NLP toolkit.It makes processing text easy!
sentences = nltk.sent_tokenize(text)
print(sentences)
```

```
['NLTK is a great NLP toolkit.It makes processing text easy!', 'Na
tural language processing primarily deals with the processing of h
uman languages using ml and dl']
```

3. Read a paragraph from the user and tokenize it into individual words using NLTK in Python.

Code:

import nltk

text = "NLTK is a great NLP toolkit. It makes processing text easy! Natural language processing primarily deals with the processing of human languages using ml and dl"

sentences = nltk.word_tokenize(text)

print(sentences)

Output:

```
import nltk
text = "NLTK is a great NLP toolkit. It makes processing text easy!
sentences = nltk.word_tokenize(text)
print(sentences)
```

```
['NLTK', 'is', 'a', 'great', 'NLP', 'toolkit', '.', 'It', 'makes',
'processing', 'text', 'easy', '!', 'Natural', 'language', 'process
ing', 'primarily', 'deals', 'with', 'the', 'processing', 'of', 'hu
man', 'languages', 'using', 'ml', 'and', 'dl']
```

4. Read a text input from the user, tokenize it into words using NLTK, and print each character of every word using Python.

Code:

```
text = "Hello World!"

words = nltk.word_tokenize(text)

print("Characters in the text:")

for word in words:

   for char in word:

      print(char)
```

Output:

```
text = "Hello World!"
words = nltk.word_tokenize(text)
print("Characters in the text:")
for word in words:
    for char in word:
        print(char)
```

```
Characters in the text:
H
e
l
l
o
W
o
r
l
d
!
```

5. Read a text input from the user and tokenize it using NLTK's WordPunctTokenizer in Python.

Code:

tokenizer = nltk.WordPunctTokenizer()

tokens=tokenizer.tokenize("Don't split contractions. E-mail: hello@example.com!")

print(tokens)

Output:

```
tokenizer = nltk.WordPunctTokenizer()
tokens=tokenizer.tokenize("Don't split contractions. E-mail: hello@example.com!")
print(tokens)
```

```
['Don', "'", 't', 'split', 'contractions', '.', 'E', '-', 'mail', ':', 'hello', '@', 'exam
ple', '.', 'com', '!']
```

6. Read a text input from the user and tokenize it using NLTK's TreebankWordTokenizer in Python.

Code:

tokenizer = nltk.TreebankWordTokenizer();

tokens = tokenizer.tokenize("Have a look at NLTK's tokenizer's.")

print(tokens)

Output:

```
tokenizer = nltk.TreebankWordTokenizer();
tokens = tokenizer.tokenize("Have a look at NLTK's tokenizer's.")
print(tokens)
```

```
['Have', 'a', 'look', 'at', 'NLTK', "'s", 'tokenizer', "'s", '.']
```

7. Read a text input from the user and tokenize it using NLTK's RegexpTokenizer to keep only words and numbers while removing punctuation.

Code:

```
tokenizer = nltk.RegexpTokenizer(r'\w+')

tokens = tokenizer.tokenize("Custom rule.keep only words & numbers,drop punctuation!")

print(tokens)
```

Output:

```
tokenizer = nltk.RegexpTokenizer(r'\w+')
tokens = tokenizer.tokenize("Custom rule.keep only words & numbers,drop punctuation!")
print(tokens)
```

```
['Custom', 'rule', 'keep', 'only', 'words', 'numbers', 'drop', 'punctuation']
```

ASSIGNMENT 2

NAME: TARLANA VIKAS SLOT: L47+L48

REGD. NO.: 23BCE7267 COURSE CODE: CSE3015

COURSE NAME: NATURAL LANGUAGE PROCESSING DATE: 18-12-2025

1. Read the paragraph from user and print the frequency of words using nltk python code.

Code:

```
import nltk

text=input("Enter your paragraph: ")

text = text.lower()  #turning into lowercase to maintain consistency

tokenizer = nltk.RegexpTokenizer(r'\w+')  #to remove punctuation and tokenization

tokens=tokenizer.tokenize(text)

freq = FreqDist(tokens);  #to find the frequency of tokens in paragraph

#to print dictionary

for item in freq.items():

   print(item)
```

Output:

```python
#Read the paragraph from user and print the frequency of words using nltk python code.
import nltk
text=input("Enter your paragraph: ")
text = text.lower()  #turning into lowercase to maintain consistency
tokenizer = nltk.RegexpTokenizer(r'\w+')  #to remove punctuation and tokenization
tokens=tokenizer.tokenize(text)
freq = FreqDist(tokens);  #to find the frequency of tokens in paragraph
#to print dictionary
for item in freq.items():
    print(item)
```

```
Enter your paragraph:  Hello everyone, this is nlp lab and i would like to implement so
me nlp projects and develop my skills.
('hello', 1)
('everyone', 1)
('this', 1)
('is', 1)
('nlp', 2)
('lab', 1)
('and', 2)
('i', 1)
('would', 1)
('like', 1)
('to', 1)
('implement', 1)
('some', 1)
('projects', 1)
('develop', 1)
('my', 1)
('skills', 1)
```

2. Read the content from a webpage and extract the tokens/ expression/ word/ number.

Code:

```
import nltk,re,requests

from nltk .probability import FreqDist

import matplotlib.pyplot as plt

url = "https://www.google.com"

text = requests.get(url).text

tokens = re.findall(r"[A-Za-z0-9]+",text.lower())

freq=FreqDist(tokens)
```

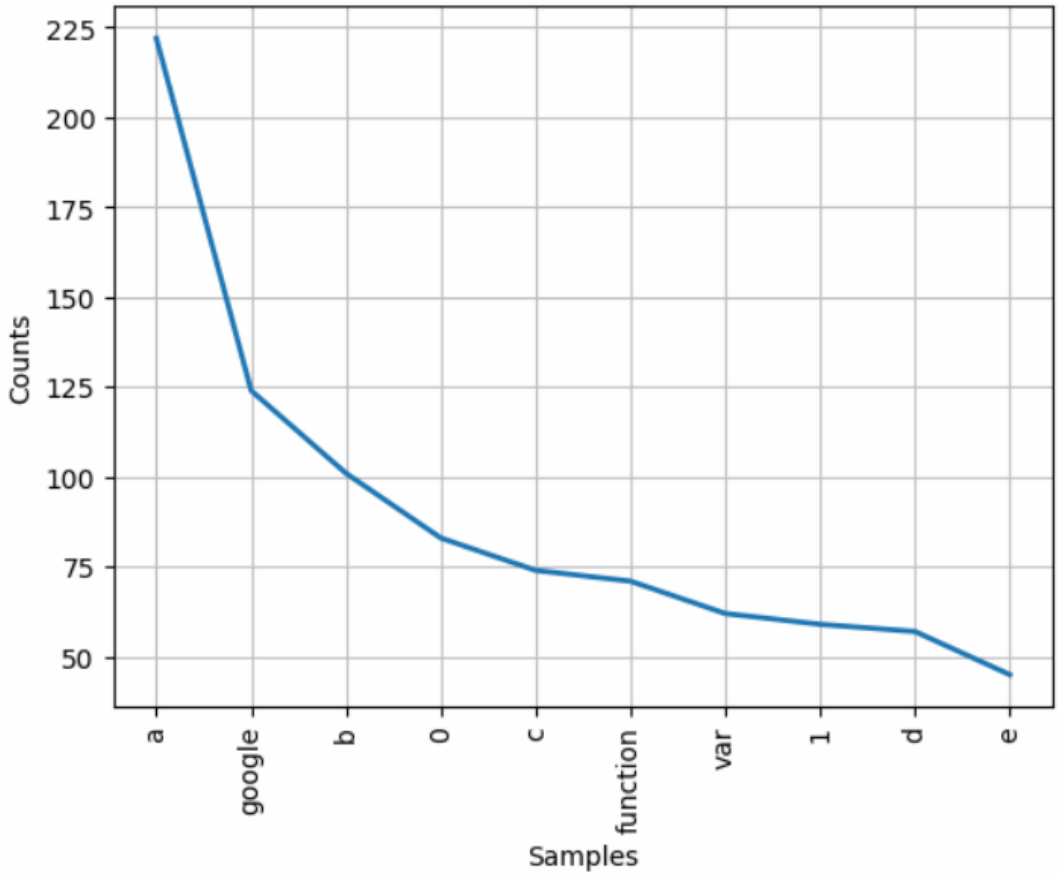
print(freq.most_common(10))

freq.plot(10)

Output:

```
#Read the content from a webpage and extract the tokens/expression/word/number
import nltk,re,requests
from nltk .probability import FreqDist
import matplotlib.pyplot as plt
url = "https://www.google.com"
text = requests.get(url).text
tokens = re.findall(r"[A-Za-z0-9]+",text.lower())
freq=FreqDist(tokens)
print(freq.most_common(10))
freq.plot(10)
```

```
[('a', 222), ('google', 124), ('b', 101), ('0', 83), ('c', 74), ('function', 71), ('var', 62), ('1', 59), ('d', 57), ('e', 45)]
```




```
<Axes: xlabel='Samples', ylabel='Counts'>
```

3. Read a list of words from the user, compute their frequency distribution using NLTK's FreqDist, and display the most common words along with a frequency plot in Python.

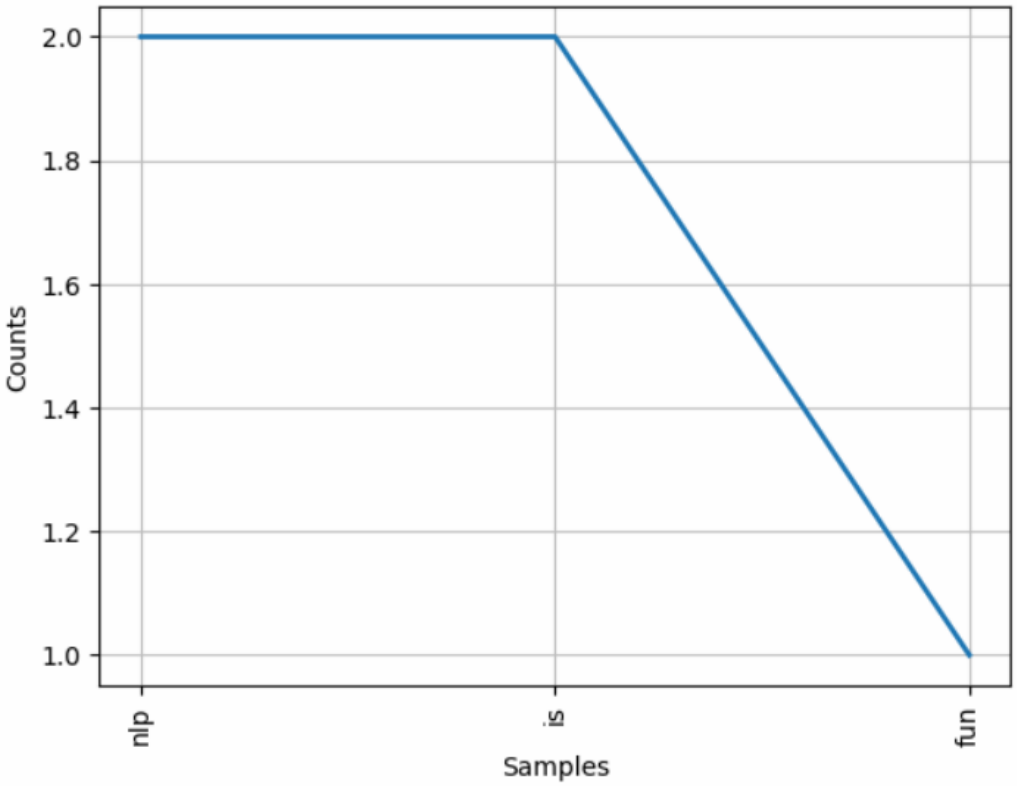
Code:

```
from nltk.probability import FreqDist

tokens = ['nlp','is','fun','nlp','is','useful']

freq = FreqDist(tokens);

print(freq)

print(freq.items())

freq.most_common(2)

print(freq['nlp'])

freq.plot(3)
```

Output:

```
from nltk.probability import FreqDist
tokens = ['nlp','is','fun','nlp','is','useful']
freq = FreqDist(tokens);
print(freq)
print(freq.items())
freq.most_common(2)
print(freq['nlp'])
freq.plot(3)
```

```
<FreqDist with 4 samples and 6 outcomes>
dict_items([('nlp', 2), ('is', 2), ('fun', 1), ('useful', 1)])
2
```




```
<Axes: xlabel='Samples', ylabel='Counts'>
```

ASSIGNMENT 3

NAME: TARLANA VIKAS SLOT: L47+L48

REGD. NO.: 23BCE7267 COURSE CODE: CSE3015

COURSE NAME: NATURAL LANGUAGE PROCESSING DATE: 08-01-2026

1. Importing stem directory using dir().

Code:

```
import nltk.stem

print(dir(nltk.stem))
```

Output:

```
import nltk.stem
print(dir(nltk.stem))
```

```
['ARLSTem', 'ARLSTem2', 'Cistem', 'ISRIStemmer', 'LancasterStemmer', 'PorterStemmer', 'RSLPStemmer', 'Regex
pStemmer', 'SnowballStemmer', 'StemmerI', 'WordNetLemmatizer', '__builtins__', '__cached__', '__doc__', '__
file__', '__loader__', '__name__', '__package__', '__path__', '__spec__', 'api', 'arlstem', 'arlstem2', 'ci
stem', 'isri', 'lancaster', 'porter', 'regexp', 'rslp', 'snowball', 'util', 'wordnet']
```

2. Using help() for stem.

Code:

```
import nltk.stem

help(nltk.stem)
```

Output:

```
import nltk.stem
help(nltk.stem)
```

```
Help on package nltk.stem in nltk:

NAME
    nltk.stem - NLTK Stemmers

DESCRIPTION
    Interfaces used to remove morphological affixes from words, leaving
    only the word stem.  Stemming algorithms aim to remove those affixes
    required for eg. grammatical role, tense, derivational morphology
    leaving only the stem of the word.  This is a difficult problem due to
    irregular words (eg. common verbs in English), complicated
    morphological rules, and part-of-speech and sense ambiguities
    (eg. ``ceil-`` is not the stem of ``ceiling``).

    StemmerI defines a standard interface for stemmers.

PACKAGE CONTENTS
    api
    arlstem
    arlstem2
    cistem
    isri
    lancaster
    porter
    regexp
    rslp
    snowball
    util
    wordnet


FILE
    c:\programdata\anaconda3\lib\site-packages\nltk\stem\__init__.py
```

3. Porter Stemmer

Code:

```
from nltk.stem import PorterStemmer #Porter Stemmer Program

stemmer = PorterStemmer()

words = ["running","studies","processing","relational","happiness"]
```

for word in words:

print(word,"->",stemmer.stem(word))

Output:

```
from nltk.stem import PorterStemmer #Porter Stemmer Program
stemmer = PorterStemmer()
words = ["running","studies","processing","relational","happiness"]
for word in words:
    print(word,"->",stemmer.stem(word))
```

```
running -> run
studies -> studi
processing -> process
relational -> relat
happiness -> happi
```

4. Snowball Stemmer Languages

Code:

```
from nltk.stem.snowball import SnowballStemmer

print(SnowballStemmer.languages)
```

Output:

```
from nltk.stem.snowball import SnowballStemmer
print(SnowballStemmer.languages)
```

```
('arabic', 'danish', 'dutch', 'english', 'finnish', 'french', 'ge
rman', 'hungarian', 'italian', 'norwegian', 'porter', 'portugues
e', 'romanian', 'russian', 'spanish', 'swedish')
```

5. Snowball Stemmer Implementation - English

Code:

```
from nltk.stem import SnowballStemmer

stemmer = SnowballStemmer("english")
```

words = ["running","studies","processing","relational","happiness"]

for word in words:

  print(word,"->",stemmer.stem(word))

Output:

```
from nltk.stem import SnowballStemmer
stemmer = SnowballStemmer("english")
words = ["running","studies","processing","relational","happiness"]
for word in words:
    print(word,"->",stemmer.stem(word))
```

```
running -> run
studies -> studi
processing -> process
relational -> relat
happiness -> happi
```

6. Snowball Stemmer Implementation - French

Code:

from nltk.stem import SnowballStemmer

stemmer = SnowballStemmer("french")

words = ["manger","mangeons","mangeais"]

for word in words:

  print(word,"->",stemmer.stem(word))

Output:

```
from nltk.stem import SnowballStemmer
stemmer = SnowballStemmer("french")
words = ["manger","mangeons","mangeais"]
for word in words:
    print(word,"->",stemmer.stem(word))
```

```
manger -> mang
mangeons -> mangeon
mangeais -> mang
```

7. Snowball Stemmer Implementation - Spanish

Code:

```
from nltk.stem import SnowballStemmer

stemmer = SnowballStemmer("spanish")

words = ["habla","hablo","hablamos","hablan"]

for word in words:

   print(word,"->",stemmer.stem(word))
```

Output:

```
from nltk.stem import SnowballStemmer
stemmer = SnowballStemmer("spanish")
words = ["habla","hablo","hablamos","hablan"]
for word in words:
    print(word,"->",stemmer.stem(word))
```

```
habla -> habl
hablo -> habl
hablamos -> habl
hablan -> habl
```

8. Lancaster Stemmer

Code:

```
from nltk.stem import LancasterStemmer

stemmer = LancasterStemmer()

words = ["running","studies","processing","relational","happiness"]

for word in words:

   print(word,"->",stemmer.stem(word))
```

Output:

```
from nltk.stem import LancasterStemmer
stemmer = LancasterStemmer()
words = ["running","studies","processing","relational","happiness"]
for word in words:
    print(word,"->",stemmer.stem(word))
```

```
running -> run
studies -> study
processing -> process
relational -> rel
happiness -> happy
```

9. Write a program to perform tokenization and stemming for the input paragraph.

Code:

```
import nltk

text=input("Enter your paragraph: ")

text = text.lower()

tokenizer = nltk.RegexpTokenizer(r'\w+')

tokens=tokenizer.tokenize(text)

print("Original Tokens")

print(tokens)

from nltk.stem import LancasterStemmer

stemmer = LancasterStemmer()

print("Stemmed Tokens")

for word in tokens:

   print(word,"->",stemmer.stem(word))
```

Output:

```python
import nltk
text=input("Enter your paragraph: ")
text = text.lower()
tokenizer = nltk.RegexpTokenizer(r'\w+')
tokens=tokenizer.tokenize(text)
print("Original Tokens")
print(tokens)
from nltk.stem import LancasterStemmer
stemmer = LancasterStemmer()
print("Stemmed Tokens")
for word in tokens:
    print(word,"->",stemmer.stem(word))
```

```
Enter your paragraph:  Today i am learning natural language processing and deep lea
rning
Original Tokens
['today', 'i', 'am', 'learning', 'natural', 'language', 'processing', 'and', 'dee
p', 'learning']
Stemmed Tokens
today -> today
i -> i
am -> am
learning -> learn
natural -> nat
language -> langu
processing -> process
and -> and
deep -> deep
learning -> learn
```

10. Word Net Lemmatizer

Code:

```
#wordnet

nltk.download("wordnet")

from nltk.stem import WordNetLemmatizer

lemmatizer = WordNetLemmatizer()

words = ["running","better","studies","cars","mice"]

for word in words:

   print(word,"->",lemmatizer.lemmatize(word))
```

Output:

```
#wordnet
nltk.download("wordnet")
```

```
from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
words = ["running","better","studies","cars","mice"]
for word in words:
    print(word,"->",lemmatizer.lemmatize(word))
```

```
running -> running
better -> better
studies -> study
cars -> car
mice -> mouse
```

11. Write a program to perform tokenization and lemmatization for the input paragraph.

Code:

```
import nltk

text=input("Enter your paragraph: ")

text = text.lower()  #turning into lowercase to maintain consistency

tokenizer = nltk.RegexpTokenizer(r'\w+')  #to remove punctuation and tokenization

tokens=tokenizer.tokenize(text)

print("Original Tokens")

print(tokens)

lemmatizer = WordNetLemmatizer()

print("Lemmatized Tokens")
```

for word in tokens:

print(word,"->",lemmatizer.lemmatize(word))

Output:

```
import nltk
text=input("Enter your paragraph: ")
text = text.lower()  #turning into lowercase to maintain consistency
tokenizer = nltk.RegexpTokenizer(r'\w+')  #to remove punctuation and tokenization
tokens=tokenizer.tokenize(text)
print("Original Tokens")
print(tokens)
lemmatizer = WordNetLemmatizer()
print("Lemmatized Tokens")
for word in tokens:
    print(word,"->",lemmatizer.lemmatize(word))
```

```
Enter your paragraph:  Today i am buying some cakes and drinks
Original Tokens
['today', 'i', 'am', 'buying', 'some', 'cakes', 'and', 'drinks']
Lemmatized Tokens
today -> today
i -> i
am -> am
buying -> buying
some -> some
cakes -> cake
and -> and
drinks -> drink
```

12. Stop Words

Code:

#Stopwords

nltk.download('stopwords')

from nltk.corpus import stopwords

stop_words = stopwords.words('english')

print(stop_words)

print("Number of stopwords: ",len(stop_words))

Output:

```
#Stopwords
nltk.download('stopwords')
```

```
from nltk.corpus import stopwords
stop_words = stopwords.words('english')
print(stop_words)
print("Number of stopwords: ",len(stop_words))
```

```
['a', 'about', 'above', 'after', 'again', 'against', 'ain', 'all', 'am', 'an', 'an
d', 'any', 'are', 'aren', "aren't", 'as', 'at', 'be', 'because', 'been', 'before',
'being', 'below', 'between', 'both', 'but', 'by', 'can', 'couldn', "couldn't", 'd',
'did', 'didn', "didn't", 'do', 'does', 'doesn', "doesn't", 'doing', 'don', "don't",
'down', 'during', 'each', 'few', 'for', 'from', 'further', 'had', 'hadn', "hadn't",
'has', 'hasn', "hasn't", 'have', 'haven', "haven't", 'having', 'he', "he'd", "he'l
l", 'her', 'here', 'hers', 'herself', "he's", 'him', 'himself', 'his', 'how', 'i',
"i'd", 'if', "i'll", "i'm", 'in', 'into', 'is', 'isn', "isn't", 'it', "it'd", "it'l
l", "it's", 'its', 'itself', "i've", 'just', 'll', 'm', 'ma', 'me', 'mightn', "migh
tn't", 'more', 'most', 'mustn', "mustn't", 'my', 'myself', 'needn', "needn't", 'n
o', 'nor', 'not', 'now', 'o', 'of', 'off', 'on', 'once', 'only', 'or', 'other', 'ou
r', 'ours', 'ourselves', 'out', 'over', 'own', 're', 's', 'same', 'shan', "shan't",
'she', "she'd", "she'll", "she's", 'should', 'shouldn', "shouldn't", "should've",
'so', 'some', 'such', 't', 'than', 'that', "that'll", 'the', 'their', 'theirs', 'th
em', 'themselves', 'then', 'there', 'these', 'they', "they'd", "they'll", "they'r
e", "they've", 'this', 'those', 'through', 'to', 'too', 'under', 'until', 'up', 'v
e', 'very', 'was', 'wasn', "wasn't", 'we', "we'd", "we'll", "we're", 'were', 'were
n', "weren't", "we've", 'what', 'when', 'where', 'which', 'while', 'who', 'whom',
'why', 'will', 'with', 'won', "won't", 'wouldn', "wouldn't", 'y', 'you', "you'd",
"you'll", 'your', "you're", 'yours', 'yourself', 'yourselves', "you've"]
Number of stopwords:  198
```

13. Write a program to remove the stopwords from the given paragraph.

Code:

```
from nltk.corpus import stopwords

text=input("Enter your paragraph: ")

text = text.lower()

tokenizer = nltk.RegexpTokenizer(r'\w+')

tokens=tokenizer.tokenize(text)
```

print("Original Tokens")

print(tokens)

stop_words = set(stopwords.words('english'))

filtered_words = [word for word in tokens if word.lower() not in stop_words]

print("Filtered Tokens")

print(filtered_words)

Output:

```
from nltk.corpus import stopwords
text=input("Enter your paragraph: ")
text = text.lower()
tokenizer = nltk.RegexpTokenizer(r'\w+')
tokens=tokenizer.tokenize(text)
print("Original Tokens")
print(tokens)
stop_words = set(stopwords.words('english'))
filtered_words = [word for word in tokens if word.lower() not in stop_words]
print("Filtered Tokens")
print(filtered_words)
```

```
Enter your paragraph:  Today i am buying some cakes and drinks in bakery
Original Tokens
['today', 'i', 'am', 'buying', 'some', 'cakes', 'and', 'drinks', 'in', 'bakery']
Filtered Tokens
['today', 'buying', 'cakes', 'drinks', 'bakery']
```

14. Write a program to print the tokens which do not repeat more than once.

Code:

from nltk.tokenize import word_tokenize

from collections import Counter

text= "Natural Language Processing makes language processing easier"

text = text.lower()

tokenizer = nltk.RegexpTokenizer(r'\w+')

tokens=tokenizer.tokenize(text)

print("Original Tokens")

print(tokens)

word_freq = Counter(tokens)

filtered_words = [word for word in tokens if word_freq[word]<2]

print("Words present only once")

print(filtered_words)

Output:

```
from nltk.tokenize import word_tokenize
from collections import Counter
text= "Natural Language Processing makes language processing easier"
text = text.lower()
tokenizer = nltk.RegexpTokenizer(r'\w+')
tokens=tokenizer.tokenize(text)
print("Original Tokens")
print(tokens)
word_freq = Counter(tokens)
filtered_words = [word for word in tokens if word_freq[word]<2]
print("Words present only once")
print(filtered_words)
```

```
Original Tokens
['natural', 'language', 'processing', 'makes', 'language', 'processing', 'easier']
Words present only once
['natural', 'makes', 'easier']
```

ASSIGNMENT 4

NAME: TARLANA VIKAS SLOT: L47+L48

REGD. NO.: 23BCE7267 COURSE CODE: CSE3015

COURSE NAME: NATURAL LANGUAGE PROCESSING DATE: 22-01-2026

1. Importing averaged_perceptron_tagger.

Code:

```
import nltk

nltk.download('averaged_perceptron_tagger');
```

Output:

```
import nltk
nltk.download('averaged_perceptron_tagger');
```

```
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     C:\Users\23BCE7267\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
```

2. POS Tagging

Code:

```
import nltk

nltk.download('punkt')

from nltk.tokenize import word_tokenize

from nltk import pos_tag


text = "A quick brown fox jumps over a black lazy dog."

text = text.lower()
```

tokens = word_tokenize(text)

print("Original Tokens")

print(tokens)

POS tagging on tokens

print("POS Tagged tokens")

pos_tags = pos_tag(tokens)

print(pos_tags)

Output:

```
import nltk
nltk.download('punkt')
from nltk.tokenize import word_tokenize
from nltk import pos_tag

text = "A quick brown fox jumps over a black lazy dog."
text = text.lower()

tokens = word_tokenize(text)
print("Original Tokens")
print(tokens)

# POS tagging on tokens
print("POS Tagged tokens")
pos_tags = pos_tag(tokens)
print(pos_tags)
```

```
Original Tokens
['a', 'quick', 'brown', 'fox', 'jumps', 'over', 'a', 'black', 'lazy', 'dog',
'.']
POS Tagged tokens
[('a', 'DT'), ('quick', 'JJ'), ('brown', 'NN'), ('fox', 'NN'), ('jumps', 'VB
Z'), ('over', 'IN'), ('a', 'DT'), ('black', 'JJ'), ('lazy', 'NN'), ('dog', 'N
N'), ('.', '.')]
```

3. Treebank

Code:

```
from nltk.corpus import treebank

#Download Dataset

nltk.download('treebank')
```

Output:

```
from nltk.corpus import treebank
#Download Dataset
nltk.download('treebank')
```

```
[nltk_data] Downloading package treebank to
[nltk_data]     C:\Users\23BCE7267\AppData\Roaming\nltk_data...
[nltk_data]   Package treebank is already up-to-date!
```

```
True
```

4. Tagged sentences in Treebank and Length

Code:

```
#Get all tagged sentences

sentences = treebank.tagged_sents()

#Count number of sentences

print("Number of sentences: ",len(sentences))
```

Output:

```
#Get all tagged sentences
sentences = treebank.tagged_sents()
#Count number of sentences
print("Number of sentences: ",len(sentences))
```

```
Number of sentences:  3914
```

5. Sentences in Treebank corpus

Code:

```
import nltk

from nltk.corpus import treebank

sentences = treebank.sents()

for i,sentence in enumerate(sentences,start=1):

    print(f"Sentence {i}:")

    print(" ".join(sentence))

    print()

#Total number of words

total_words = sum(len(sentence) for sentence in sentences)

print("Number of words",total_words)
```

Output:

```
import nltk
from nltk.corpus import treebank
sentences = treebank.sents()
for i,sentence in enumerate(sentences,start=1):
    print(f"Sentence {i}:")
    print(" ".join(sentence))
    print()
#Total number of words
total_words = sum(len(sentence) for sentence in sentences)
print("Number of words",total_words)
```

```
Sentence 1:
Pierre Vinken , 61 years old , will join the board as a nonexecutive directo
r Nov. 29 .

Sentence 2:
Mr. Vinken is chairman of Elsevier N.V. , the Dutch publishing group .

Sentence 3:
Rudolph Agnew , 55 years old and former chairman of Consolidated Gold Fields
PLC , was named *-1 a nonexecutive director of this British industrial congl
omerate .

Sentence 4:
A form of asbestos once used * * to make Kent cigarette filters has caused a
high percentage of cancer deaths among a group of workers exposed * to it mo
re than 30 years ago , researchers reported 0 *T*-1 .
```

6. Display first 10 POS Tags.

Code:

```
#Display first 10 POS Tags

sentences = treebank.tagged_sents()

for i,sentence in enumerate(sentences[:10],start=1):

    print(f"Sentence {i}:")

    print(sentence)

    print()
```

Output:

```
#Display first 10 POS Tags
sentences = treebank.tagged_sents()
for i,sentence in enumerate(sentences[:10],start=1):
    print(f"Sentence {i}:")
    print(sentence)
    print()
```

```
Sentence 1:
[('Pierre', 'NNP'), ('Vinken', 'NNP'), (',', ','), ('61', 'CD'), ('years', 'NN
S'), ('old', 'JJ'), (',', ','), ('will', 'MD'), ('join', 'VB'), ('the', 'DT'),
('board', 'NN'), ('as', 'IN'), ('a', 'DT'), ('nonexecutive', 'JJ'), ('directo
r', 'NN'), ('Nov.', 'NNP'), ('29', 'CD'), ('.', '.')]

Sentence 2:
[('Mr.', 'NNP'), ('Vinken', 'NNP'), ('is', 'VBZ'), ('chairman', 'NN'), ('of',
'IN'), ('Elsevier', 'NNP'), ('N.V.', 'NNP'), (',', ','), ('the', 'DT'), ('Dutc
h', 'NNP'), ('publishing', 'VBG'), ('group', 'NN'), ('.', '.')]
```

7. Unigram Tagger

Code:

```
#Unigram Tagger

import nltk

from nltk.corpus import treebank

from nltk.tag import UnigramTagger

from nltk.tokenize import word_tokenize

#Training data

train_data = treebank.tagged_sents()[:2000]

#Create Unigram Tagger

unigram_tagger = UnigramTagger(train_data)

#Test Sentence

sentence = "A quick brown fox jumps over a black lazy dog."

tokens = word_tokenize(sentence)

#POS Tagging

print(unigram_tagger.tag(tokens))
```

Output:

```
#Unigram Tagger
import nltk
from nltk.corpus import treebank
from nltk.tag import UnigramTagger
from nltk.tokenize import word_tokenize
#Training data
train_data = treebank.tagged_sents()[:2000]
#Create Unigram Tagger
unigram_tagger = UnigramTagger(train_data)
#Test Sentence
sentence = "A quick brown fox jumps over a black lazy dog."
tokens = word_tokenize(sentence)
#POS Tagging
print(unigram_tagger.tag(tokens))
```

```
[('A', 'DT'), ('quick', 'JJ'), ('brown', None), ('fox', None), ('jumps', Non
e), ('over', 'IN'), ('a', 'DT'), ('black', 'JJ'), ('lazy', None), ('dog', Non
e), ('.', '.')]
```

8. BI-GRAM Tagger

Code:

#BI-GRAM Tagger

import nltk

from nltk.corpus import treebank

from nltk.tag import BigramTagger,UnigramTagger

from nltk.tokenize import word_tokenize

#Load Training data

train_data = treebank.tagged_sents()[:2000]

#Create Unigram Tagger

unigram_tagger = UnigramTagger(train_data)

#Create Bigram Tagger

bigram_tagger = BigramTagger(train_data,backoff=unigram_tagger)

#Test Sentence

sentence = "A quick brown fox jumps over a black lazy dog."

tokens = word_tokenize(sentence)

#POS Tagging using Bigram tagger

print(bigram_tagger.tag(tokens))

Output:

```
#BI-GRAM Tagger
import nltk
from nltk.corpus import treebank
from nltk.tag import BigramTagger,UnigramTagger
from nltk.tokenize import word_tokenize
#Load Training data
train_data = treebank.tagged_sents()[:2000]
#Create Unigram Tagger
unigram_tagger = UnigramTagger(train_data)
#Create Bigram Tagger
bigram_tagger = BigramTagger(train_data,backoff=unigram_tagger)
#Test Sentence
sentence = "A quick brown fox jumps over a black lazy dog."
tokens = word_tokenize(sentence)
#POS Tagging using Bigram tagger
print(bigram_tagger.tag(tokens))
```

```
[('A', 'DT'), ('quick', 'JJ'), ('brown', None), ('fox', None), ('jumps', Non
e), ('over', 'IN'), ('a', 'DT'), ('black', 'JJ'), ('lazy', None), ('dog', Non
e), ('.', '.')]
```

9. Brown dataset

Code:

#Brown dataset

import nltk

from nltk.corpus import brown

nltk.download('brown')

Output:

```
#Brown dataset
import nltk
from nltk.corpus import brown
nltk.download('brown')
```

```
[nltk_data] Downloading package brown to
[nltk_data]     C:\Users\23BCE7267\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping corpora\brown.zip.
True
```

10. Number of Sentences in Brown Dataset

Code:

```
#Number of Sentences

print("Number of sentences",len(brown.sents()))

#Display Sentences

for sent in brown.sents()[:3]:

    print(" ".join(sent))

#Tagged sentences(POS Tagged)

for sent in brown.tagged_sents()[:3]:

    print(sent)

#Words in a specific Category

brown.words(categories='news')[:20]
```

Output:

```
#Number of Sentences
print("Number of sentences",len(brown.sents()))
#Display Sentences
for sent in brown.sents()[:3]:
    print(" ".join(sent))
#Tagged sentences(POS Tagged)
for sent in brown.tagged_sents()[:3]:
    print(sent)
#Words in a specific Category
brown.words(categories='news')[:20]
```

```
Number of sentences 57340
The Fulton County Grand Jury said Friday an investigation of Atlanta's recen
t primary election produced `` no evidence '' that any irregularities took p
lace .
The jury further said in term-end presentments that the City Executive Commi
ttee , which had over-all charge of the election , `` deserves the praise an
d thanks of the City of Atlanta '' for the manner in which the election was
conducted .
The September-October term jury had been charged by Fulton Superior Court Ju
dge Durwood Pye to investigate reports of possible `` irregularities '' in t
he hard-fought primary which was won by Mayor-nominate Ivan Allen Jr. .
[('The', 'AT'), ('Fulton', 'NP-TL'), ('County', 'NN-TL'), ('Grand', 'JJ-T
L'), ('Jury', 'NN-TL'), ('said', 'VBD'), ('Friday', 'NR'), ('an', 'AT'), ('i
nvestigation', 'NN'), ('of', 'IN'), ("Atlanta's", 'NP$'), ('recent', 'JJ'),
('primary', 'NN'), ('election', 'NN'), ('produced', 'VBD'), ('``', '``'),
('no', 'AT'), ('evidence', 'NN'), ("''", "''"), ('that', 'CS'), ('any', 'DT
I'), ('irregularities', 'NNS'), ('took', 'VBD'), ('place', 'NN'), ('.',
'.')]
```

11. TRI-GRAM Tagger

Code:

TRI-GRAM Tagger

import nltk

from nltk.corpus import treebank

from nltk.tag import BigramTagger, UnigramTagger, TrigramTagger

from nltk.tokenize import word_tokenize

```
# Load Training data
train_data = treebank.tagged_sents()[:2000]

# Create Unigram Tagger
unigram_tagger = UnigramTagger(train_data)

# Create Bigram Tagger with backoff to Unigram
bigram_tagger = BigramTagger(train_data, backoff=unigram_tagger)

# Create Trigram Tagger with backoff to Bigram
trigram_tagger = TrigramTagger(train_data, backoff=bigram_tagger)

# Test Sentence
sentence = "A quick brown fox jumps over a black lazy dog."
tokens = word_tokenize(sentence)

# POS Tagging using Trigram tagger
print(trigram_tagger.tag(tokens))
```

Output:

```python
# TRI-GRAM Tagger
import nltk
from nltk.corpus import treebank
from nltk.tag import BigramTagger, UnigramTagger, TrigramTagger
from nltk.tokenize import word_tokenize

# Load Training data
train_data = treebank.tagged_sents()[:2000]

# Create Unigram Tagger
unigram_tagger = UnigramTagger(train_data)

# Create Bigram Tagger with backoff to Unigram
bigram_tagger = BigramTagger(train_data, backoff=unigram_tagger)

# Create Trigram Tagger with backoff to Bigram
trigram_tagger = TrigramTagger(train_data, backoff=bigram_tagger)

# Test Sentence
sentence = "A quick brown fox jumps over a black lazy dog."
tokens = word_tokenize(sentence)

# POS Tagging using Trigram tagger
print(trigram_tagger.tag(tokens))
```

```
[('A', 'DT'), ('quick', 'JJ'), ('brown', None), ('fox', None), ('jumps', Non
e), ('over', 'IN'), ('a', 'DT'), ('black', 'JJ'), ('lazy', None), ('dog', Non
e), ('.', '.')]
```

12. BI-GRAM Tagger Accuracy

Code:

#BI-GRAM Tagger

import nltk

from nltk.corpus import treebank

from nltk.tag import BigramTagger,UnigramTagger

from nltk.tokenize import word_tokenize

sentences = treebank.tagged_sents()

train_data = sentences[:2000]

test_data = sentences[1000:]

unigram_tagger = UnigramTagger(train_data)

bigram_tagger = BigramTagger(train_data,backoff=unigram_tagger)

accuracy = bigram_tagger.evaluate(test_data)

print("Bigram Tagger Accuracy: ",accuracy)

sentence = "NLP is very interesting."

tokens=word_tokenize(sentence)

print(bigram_tagger.tag(tokens))

Output:

```
#BI-GRAM Tagger
import nltk
from nltk.corpus import treebank
from nltk.tag import BigramTagger,UnigramTagger
from nltk.tokenize import word_tokenize
sentences = treebank.tagged_sents()
train_data = sentences[:2000]
test_data = sentences[1000:]
unigram_tagger = UnigramTagger(train_data)
bigram_tagger = BigramTagger(train_data,backoff=unigram_tagger)
accuracy = bigram_tagger.evaluate(test_data)
print("Bigram Tagger Accuracy: ",accuracy)
sentence = "NLP is very interesting."
tokens=word_tokenize(sentence)
print(bigram_tagger.tag(tokens))
```

```
C:\Users\23BCE7267\AppData\Local\Temp\ipykernel_356\3023607440.py:11: Deprecat
ionWarning:
  Function evaluate() has been deprecated.  Use accuracy(gold)
  instead.
  accuracy = bigram_tagger.evaluate(test_data)
Bigram Tagger Accuracy:  0.8887696709585121
[('NLP', None), ('is', 'VBZ'), ('very', 'RB'), ('interesting', 'JJ'), ('.',
'.')]
```

ASSIGNMENT 5

NAME: TARLANA VIKAS SLOT: L47+L48

REGD. NO.: 23BCE7267 COURSE CODE: CSE3015

COURSE NAME: NATURAL LANGUAGE PROCESSING DATE: 29-01-2026

1. Importing nltk and punkt.

Code:

```
import nltk

nltk.download('punkt');
```

Output:

```
import nltk
```

```
nltk.download('punkt')
```

```
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\23bce7267\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
```

```
True
```

2. Regex Tagger

Code:

```
#Regex Tagger

from nltk.tag import RegexpTagger

#Define regex pattern

patterns = [

   (r'.*ing$','VBG'),

   (r'.*ed$','VBD'),
```

```
    (r'.*es$','VBZ'),
    (r'.*s$','NNS'),
    (r'^[0-9]+$','CD'),
    (r'.*','NN')
]
regex_tagger = RegexpTagger(patterns)
sentence = "Dogs are running quickly"
tokens = nltk.word_tokenize(sentence)
tags = regex_tagger.tag(tokens)
print(tags)
```

Output:

```
#Regex Tagger
from nltk.tag import RegexpTagger
#Define regex pattern
patterns = [
    (r'.*ing$','VBG'),
    (r'.*ed$','VBD'),
    (r'.*es$','VBZ'),
    (r'.*s$','NNS'),
    (r'^[0-9]+$','CD'),
    (r'.*','NN')
]
regex_tagger = RegexpTagger(patterns)
sentence = "Dogs are running quickly"
tokens = nltk.word_tokenize(sentence)
tags = regex_tagger.tag(tokens)
print(tags)
```

```
[('Dogs', 'NNS'), ('are', 'NN'), ('running', 'VBG'), ('quickly', 'NN')]
```

3. Write a Python script to tag parts of speech in the given sentence. “70

students are attending Natural language processing lab and they completed all the programs quickly." Define patterns for pronouns, conjunctions, prepositions, determiners, adjectives, verbs, adverbs, and nouns. Then, print the tagged words

Code:

```
from nltk.tag import RegexpTagger


patterns = [

(r'^\d+$', 'CD'), # Cardinal numbers

(r'\b(I|me|my|he|him|his|she|her|we|you|it|they)\b', 'PRP'), # pronouns

(r'\b(on|in|at|by|with|about|into|to)\b', 'IN'), #prepositions

(r'\b(and|or|but|also)\b', 'CC'), #Conjunctions

(r'\b(The|the|A|a|An|an)\b', 'DT'), # Determiners

(r'\b\w+able\b', 'JJ'), # Adjectives ending in 'able'

(r'.*ing$', 'VBG'), # Gerunds

(r'.*ed$', 'VBD'), # Past tense verbs

(r'.*es$', 'VBZ'), # 3rd person singular verbs

(r'.*ly$', 'RB'), # Adverbs

(r'.*', 'NN') # Default: Noun

]

regex_tagger = RegexpTagger(patterns)

sentence = "70 students are attending Natural language processing lab and
they completed all the programs quickly."
```

tokens = nltk.word_tokenize(sentence)

tags = regex_tagger.tag(tokens)

print(tags)

Output:

```
from nltk.tag import RegexpTagger

patterns = [
(r'^\d+$', 'CD'), # Cardinal numbers
(r'\b(I|me|my|he|him|his|she|her|we|you|it|they)\b', 'PRP'), # pronouns
(r'\b(on|in|at|by|with|about|into|to)\b', 'IN'), #prepositions
(r'\b(and|or|but|also)\b', 'CC'), #Conjunctions
(r'\b(The|the|A|a|An|an)\b', 'DT'), # Determiners
(r'\b\w+able\b', 'JJ'), # Adjectives ending in 'able'
(r'.*ing$', 'VBG'), # Gerunds
(r'.*ed$', 'VBD'), # Past tense verbs
(r'.*es$', 'VBZ'), # 3rd person singular verbs
(r'.*ly$', 'RB'), # Adverbs
(r'.*', 'NN') # Default: Noun
]
regex_tagger = RegexpTagger(patterns)
sentence = "70 students are attending Natural language processing lab and they completed all the programs quickly."
tokens = nltk.word_tokenize(sentence)
tags = regex_tagger.tag(tokens)
print(tags)
```

```
[('70', 'CD'), ('students', 'NN'), ('are', 'NN'), ('attending', 'VBG'), ('Natural', 'NN'), ('language', 'NN'), ('processing',
'VBG'), ('lab', 'NN'), ('and', 'CC'), ('they', 'PRP'), ('completed', 'VBD'), ('all', 'NN'), ('the', 'DT'), ('programs', 'NN'),
('quickly', 'RB'), ('.', 'NN')]
```

4. NER (Name Entity Recognition) Tagger

Code:

#NER (Name Entity Recognition) Tagger

import nltk

import nltk

nltk.download('punkt')

nltk.download('averaged_perceptron_tagger')

nltk.download('maxent_ne_chunker')

nltk.download('maxent_ne_chunker_tab')

nltk.download('words')

sentence = "Barack Obama was born in Hawaii"

tokens = nltk.word_tokenize(sentence)

pos_tags = nltk.pos_tag(tokens)

tree = nltk.ne_chunk(pos_tags)

print(tree)

Output:

```
In [12]: #NER (Name Entity Recognition) Tagger
         import nltk
         import nltk
         nltk.download('punkt')
         nltk.download('averaged_perceptron_tagger')
         nltk.download('maxent_ne_chunker')
         nltk.download('maxent_ne_chunker_tab')
         nltk.download('words')
         sentence = "Barack Obama was born in Hawaii"
         tokens = nltk.word_tokenize(sentence)
         pos_tags = nltk.pos_tag(tokens)
         tree = nltk.ne_chunk(pos_tags)
         print(tree)

[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\23bce7267\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     C:\Users\23bce7267\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping taggers\averaged_perceptron_tagger.zip.
[nltk_data] Downloading package maxent_ne_chunker to
[nltk_data]     C:\Users\23bce7267\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping chunkers\maxent_ne_chunker.zip.
[nltk_data] Downloading package maxent_ne_chunker_tab to
[nltk_data]     C:\Users\23bce7267\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping chunkers\maxent_ne_chunker_tab.zip.
[nltk_data] Downloading package words to
[nltk_data]     C:\Users\23bce7267\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping corpora\words.zip.

(S
  (PERSON Barack/NNP)
  (PERSON Obama/NNP)
  was/VBD
  born/VBN
  in/IN
  (GPE Hawaii/NNP))
```

5. NER Tagging example

Code:

sentence = "70 students are attending Natural language processing lab and they completed all the programs quickly."

tokens = nltk.word_tokenize(sentence)

pos_tags = nltk.pos_tag(tokens)

tree = nltk.ne_chunk(pos_tags)

print(tree)

Output:

```
sentence = "70 students are attending Natural language processing lab and they completed all the programs quickly."
tokens = nltk.word_tokenize(sentence)
pos_tags = nltk.pos_tag(tokens)
tree = nltk.ne_chunk(pos_tags)
print(tree)
```

```
(S
  70/CD
  students/NNS
  are/VBP
  attending/VBG
  (ORGANIZATION Natural/NNP)
  language/NN
  processing/NN
  lab/NN
  and/CC
  they/PRP
  completed/VBD
  all/PDT
  the/DT
  programs/NNS
  quickly/RB
  ./.)
```

6. Installing spacy. (In Google Colab)

Code:

!pip install spacy

!python -m spacy download en_core_web_sm

Output:

```
!pip install spacy
!python -m spacy download en_core_web_sm
```

```
Requirement already satisfied: spacy in /usr/local/lib/python3.12/dist-packages (3.8.11)
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.0.12)
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.0.5)
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.0.15)
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.13)
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.0.12)
Requirement already satisfied: thinc<8.4.0,>=8.3.4 in /usr/local/lib/python3.12/dist-packages (from spacy) (8.3.10)
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.1.3)
Requirement already satisfied: srsly<3.0.0,>=2.4.3 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.5.2)
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.10)
Requirement already satisfied: weasel<0.5.0,>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (0.4.3)
Requirement already satisfied: typer-slim<1.0.0,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (0.21.1)
Requirement already satisfied: tqdm<5.0.0,>=4.38.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (4.67.1)
Requirement already satisfied: numpy>=1.19.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.2)
Requirement already satisfied: requests<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.32.4)
Requirement already satisfied: pydantic!=1.8,!=1.8.1,<3.0.0,>=1.7.4 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.12.3)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.1.6)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from spacy) (75.2.0)
```

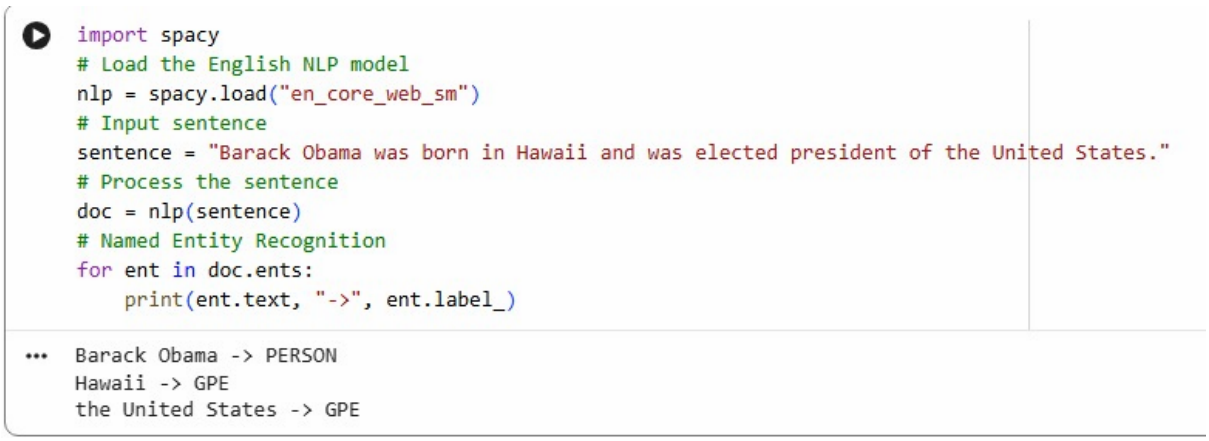
7. Using Spacy (In Google Colab)

Code:

import spacy

```
# Load the English NLP model

nlp = spacy.load("en_core_web_sm")

# Input sentence

sentence = "Barack Obama was born in Hawaii and was elected president of
the United States."

# Process the sentence

doc = nlp(sentence)

# Named Entity Recognition

for ent in doc.ents:

    print(ent.text, "->", ent.label_)
```

Output:



ASSIGNMENT 6

NAME: TARLANA VIKAS SLOT: L47+L48

REGD. NO.: 23BCE7267 COURSE CODE: CSE3015

COURSE NAME: NATURAL LANGUAGE PROCESSING DATE: 12-02-2026

1. Implementing Brill Tagger.

Code:

```
#Brill Tagger


import nltk
from nltk.tag import UnigramTagger
from nltk.tag.brill import Pos
from nltk.tag.brill_trainer import BrillTaggerTrainer
#Training Data
train = [
    [("I","PRP"),("run","VBP")],
    [("He","PRO"),("runs","VBZ")]
]
#Baseline Tagger
baseline = UnigramTagger(train)
#Templates
templates = [Pos([-1]),Pos([-1])]
```

#Train Brill Tagger

trainer = BrillTaggerTrainer(baseline,templates)

brill = trainer.train(train)

#Test

print(brill.tag(["He","run"]))

Output:

```
#Brill Tagger

import nltk
from nltk.tag import UnigramTagger
from nltk.tag.brill import Pos
from nltk.tag.brill_trainer import BrillTaggerTrainer
#Training Data
train = [
    [("I","PRP"),("run","VBP")],
    [("He","PRO"),("runs","VBZ")]
]
#Baseline Tagger
baseline = UnigramTagger(train)
#Templates
templates = [Pos([-1]),Pos([-1])]

#Train Brill Tagger
trainer = BrillTaggerTrainer(baseline,templates)
brill = trainer.train(train)

#Test
print(brill.tag(["He","run"]))
```

```
[('He', 'PRO'), ('run', 'VBP')]
```

2. Implementing Brill Tagger using large training data and different testing data.

Code:

```
#Brill Tagger with different training and testing data


import nltk

from nltk.tag import UnigramTagger

from nltk.tag.brill import Pos

from nltk.tag.brill_trainer import BrillTaggerTrainer

#Training Data

train = [

   [("John","NNP"),("plays","VBZ"),("cricket","NN")],

   [("Mary","NNP"),("plays","VBZ"),("football","NN")],

   [("The","DT"),("dog","NN"),("barks","VBZ")],

   [("A","DT"),("cat","NN"),("runs","VBZ")]

]

#Baseline Tagger

baseline = UnigramTagger(train)

#Templates

templates = [Pos([-1]),Pos([-1])]


#Train Brill Tagger

trainer = BrillTaggerTrainer(baseline,templates)
```

brill = trainer.train(train)

#Test

print(brill.tag(["Mary","runs"]))

Output:

```
#Brill Tagger with different training and testing data

import nltk
from nltk.tag import UnigramTagger
from nltk.tag.brill import Pos
from nltk.tag.brill_trainer import BrillTaggerTrainer
#Training Data
train = [
    [("John","NNP"),("plays","VBZ"),("cricket","NN")],
    [("Mary","NNP"),("plays","VBZ"),("football","NN")],
    [("The","DT"),("dog","NN"),("barks","VBZ")],
    [("A","DT"),("cat","NN"),("runs","VBZ")]
]
#Baseline Tagger
baseline = UnigramTagger(train)
#Templates
templates = [Pos([-1]),Pos([-1])]

#Train Brill Tagger
trainer = BrillTaggerTrainer(baseline,templates)
brill = trainer.train(train)

#Test
print(brill.tag(["Mary","runs"]))
```

```
[('Mary', 'NNP'), ('runs', 'VBZ')]
```

3. Implementing Date Regexp Tagger

Code:

```
#Date Regexp Tagger


import nltk

from nltk.tokenize import word_tokenize

from nltk.tag import RegexpTagger


#Define patterns (order matters)

patterns=[

   (r'\d{1,2}/\d{1,2}/\d{2,4}','DATE'), # 12/05/2024 Numeric Format

   (r'\d{4}-\d{2}-\d{2}','DATE'), # 2024-08-10 ISO Format

(r'(?i)january|february|march|april|may|june|july|august|september|octobe
r|november|december','DATE'), #Months

   (r'.*','O') #Default tag

]


tagger = RegexpTagger(patterns)

sentence = "Meeting on 12/05/2024 and March"

tokens = word_tokenize(sentence)

print(tagger.tag(tokens))
```

Output:

```python
#Date Regexp Tagger

import nltk
from nltk.tokenize import word_tokenize
from nltk.tag import RegexpTagger

#Define patterns (order matters)
patterns=[
    (r'\d{1,2}/\d{1,2}/\d{2,4}','DATE'), # 12/05/2024 Numeric Format
    (r'\d{4}-\d{2}-\d{2}','DATE'), # 2024-08-10 ISO Format
    (r'(?i)january|february|march|april|may|june|july|august|september|october|november|december','DATE'), #Months
    (r'.*','O') #Default tag
]

tagger = RegexpTagger(patterns)
sentence = "Meeting on 12/05/2024 and March"
tokens = word_tokenize(sentence)
print(tagger.tag(tokens))
```

```
[('Meeting', 'O'), ('on', 'O'), ('12/05/2024', 'DATE'), ('and', 'O'), ('March', 'DATE')]
```

4. Implementing Money Tagger

Code:

#Money Tagger

```
import nltk

from nltk.tokenize import word_tokenize

from nltk.tag import RegexpTagger


#Define money patterns

patterns = [

   (r'[$  ]\d+(\.\d+)?','MONEY'), #$200,   800, $20.50

   (r'\d+(\.\d+)$','MONEY'), #200$

   (r'(?i)(dollars|rupees|usd|inr)','MONEY'), #currency words

   (r'.*','O') #default

]
```

tagger = RegexpTagger(patterns)

sentence = "I paid 500 and 200 dollars for the book."

tokens = word_tokenize(sentence)

print(tagger.tag(tokens))

Output:

```
#Money Tagger

import nltk
from nltk.tokenize import word_tokenize
from nltk.tag import RegexpTagger

#Define money patterns
patterns = [
    (r'[$₹]\d+(\.\d+)?','MONEY'), #$200, ₹800, $20.50
    (r'\d+(\.\d+)$','MONEY'), #200$
    (r'(?i)(dollars|rupees|usd|inr)','MONEY'), #currency words
    (r'.*','O') #default
]

tagger = RegexpTagger(patterns)
sentence = "I paid ₹500 and 200 dollars for the book."
tokens = word_tokenize(sentence)
print(tagger.tag(tokens))
```

```
[('I', 'O'), ('paid', 'O'), ('₹500', 'MONEY'), ('and', 'O'), ('200', 'O'), ('dollars', 'MONEY'), ('for', 'O'), ('the', 'O'), ('book', 'O'), ('.', 'O')]
```

5. Practice: Test the sentence "I paid 500 on 12/05/2024 and $200 in March."

Code:

#Practice: Test the sentence "I paid 500 on 12/05/2024 and $200 in March."

import nltk

from nltk.tokenize import word_tokenize

from nltk.tag import RegexpTagger

#Define money and date patterns

patterns = [

  (r'[$]\d+(\.\d+)?','MONEY'), #$200, 800, $20.50

```
    (r'\d+(\.\d+)$','MONEY'), #200$
    (r'(?i)(dollars|rupees|usd|inr)','MONEY'), #currency words
    (r'\d{1,2}/\d{1,2}/\d{2,4}','DATE'), # 12/05/2024 Numeric Format
    (r'\d{4}-\d{2}-\d{2}','DATE'), # 2024-08-10 ISO Format
(r'(?i)january|february|march|april|may|june|july|august|september|octobe
r|november|december','DATE'), #Months
    (r'.*','O') #default
]
```

```
tagger = RegexpTagger(patterns)
sentence = "I paid  500 on 12/05/2024 and $200 in March."
tokens = word_tokenize(sentence)
print(tagger.tag(tokens))
```

Output:

```
#Practice: Test the sentence "I paid ₹500 on 12/05/2024 and $200 in March."

import nltk
from nltk.tokenize import word_tokenize
from nltk.tag import RegexpTagger

#Define money and date patterns
patterns = [
    (r'[$₹]\d+(\.\d+)?','MONEY'), #$200, ₹800, $20.50
    (r'\d+(\.\d+)$','MONEY'), #200$
    (r'(?i)(dollars|rupees|usd|inr)','MONEY'), #currency words
    (r'\d{1,2}/\d{1,2}/\d{2,4}','DATE'), # 12/05/2024 Numeric Format
    (r'\d{4}-\d{2}-\d{2}','DATE'), # 2024-08-10 ISO Format
    (r'(?i)january|february|march|april|may|june|july|august|september|october|november|december','DATE'), #Months
    (r'.*','O') #default
]

tagger = RegexpTagger(patterns)
sentence = "I paid ₹500 on 12/05/2024 and $200 in March."
tokens = word_tokenize(sentence)
print(tagger.tag(tokens))
```

```
[('I', 'O'), ('paid', 'O'), ('₹500', 'MONEY'), ('on', 'O'), ('12/05/2024', 'DATE'), ('and', 'O'), ('$', 'O'), ('200', 'O'), ('in', 'O'), ('March', 'DAT
E'), ('.', 'O')]
```

6. Implementing Maximum Entropy Classifier(MEC) Tagger.

Code:

```
#Maximum Entropy Classifier(MEC) Tagger
```

```
import nltk
from nltk.classify import MaxentClassifier

#Training Data
train_data = [
    ({"word":"good"},"Positive"),
    ({"word":"excellent"},"Positive"),
    ({"word":"amazing"},"Positive"),
    ({"word":"bad"},"Negative"),
    ({"word":"poor"},"Negative"),
    ({"word":"terrible"},"Negative")
]

#Train Maximum Entropy Classifier
classifier = MaxentClassifier.train(train_data,max_iter=5)

#Test the Classifier
test = {"word":"good"}
print(classifier.classify(test))

test2 = {"word":"bad"}
print(classifier.classify(test2))
```

Output:

```
#Maximum Entropy Classifier(MEC) Tagger
import nltk
from nltk.classify import MaxentClassifier

#Training Data
train_data = [
    ({"word":"good"},"Positive"),
    ({"word":"excellent"},"Positive"),
    ({"word":"amazing"},"Positive"),
    ({"word":"bad"},"Negative"),
    ({"word":"poor"},"Negative"),
    ({"word":"terrible"},"Negative")
]

#Train Maximum Entropy Classifier
classifier = MaxentClassifier.train(train_data,max_iter=5)

#Test the Classifier
test = {"word":"good"}
print(classifier.classify(test))

test2 = {"word":"bad"}
print(classifier.classify(test2))
```

```
  ==> Training (5 iterations)

      Iteration    Log Likelihood    Accuracy
      ---------------------------------------
             1          -0.69315       0.500
             2          -0.40547       1.000
             3          -0.28768       1.000
             4          -0.22314       1.000
         Final          -0.18232       1.000
Positive
Negative
```

ASSIGNMENT 7

NAME: TARLANA VIKAS SLOT: L47+L48

REGD. NO.: 23BCE7267 COURSE CODE: CSE3015

COURSE NAME: NATURAL LANGUAGE PROCESSING DATE: 19-02-2026

1. Implementing Context Free Grammar.

Code:

```
#Grammar


import nltk

from nltk import CFG

from nltk.parse.generate import generate


#Define the grammar

grammar = CFG.fromstring("""

                S -> NP VP

                NP -> Det N

                VP -> V NP

                Det -> 'a' | 'the'

                N ->'cat' | 'dog'

                V ->'chased'|'saw'

                """)

#Generate sentences
```

for sentence in generate(grammar):

print(' '.join(sentence))

Output:

```
#Grammar
import nltk
from nltk import CFG
from nltk.parse.generate import generate
#Define the grammar
grammar = CFG.fromstring("""
                         S -> NP VP
                         NP -> Det N
                         VP -> V NP
                         Det -> 'a' | 'the'
                         N ->'cat' | 'dog'
                         V ->'chased'|'saw'
                         """)
#Generate sentences
for sentence in generate(grammar):
    print(' '.join(sentence))
```

```
a cat chased a cat
a cat chased a dog
a cat chased the cat
a cat chased the dog
a cat saw a cat
a cat saw a dog
a cat saw the cat
a cat saw the dog
a dog chased a cat
a dog chased a dog
a dog chased the cat
a dog chased the dog
a dog saw a cat
a dog saw a dog
a dog saw the cat
a dog saw the dog
the cat chased a cat
the cat chased a dog
the cat chased the cat
the cat chased the dog
the cat saw a cat
the cat saw a dog
the cat saw the cat
the cat saw the dog
the dog chased a cat
the dog chased a dog
the dog chased the cat
the dog chased the dog
the dog saw a cat
the dog saw a dog
the dog saw the cat
the dog saw the dog
```

2. Implementing Context Free Grammar example 2.

Code:

```
#Grammar example 2
import nltk
from nltk import CFG
from nltk.parse.generate import generate
#Define the grammar
grammar = CFG.fromstring("""
                    S -> NP VP
                    NP -> N | ART N
                    VP -> AUX V NP
                    N -> 'Rahul' | 'apple'
                    AUX -> 'is'
                    V -> 'eating'
                    ART -> 'an'
                    """)
#Generate sentences
for sentence in generate(grammar):
   print(' '.join(sentence))
```

Output:

```
#Grammar example 2
import nltk
from nltk import CFG
from nltk.parse.generate import generate
#Define the grammar
grammar = CFG.fromstring("""
                         S -> NP VP
                         NP -> N | ART N
                         VP -> AUX V NP
                         N -> 'Rahul' | 'apple'
                         AUX -> 'is'
                         V -> 'eating'
                         ART -> 'an'
                         """)
#Generate sentences
for sentence in generate(grammar):
    print(' '.join(sentence))
```

```
Rahul is eating Rahul
Rahul is eating apple
Rahul is eating an Rahul
Rahul is eating an apple
apple is eating Rahul
apple is eating apple
apple is eating an Rahul
apple is eating an apple
an Rahul is eating Rahul
an Rahul is eating apple
an Rahul is eating an Rahul
an Rahul is eating an apple
an apple is eating Rahul
an apple is eating apple
an apple is eating an Rahul
an apple is eating an apple
```

3. Implementing Shallow Parsing.

Code:

Shallow Parsing

import nltk

from nltk import word_tokenize, pos_tag

from nltk.chunk import RegexpParser

text = "The small boy is playing in the garden"

tags = pos_tag(word_tokenize(text))

grammar = "NP:{<DT>?<JJ>*<NN>}"

parser = RegexpParser(grammar)

result = parser.parse(tags)

print(result)

result.draw

Output:

```
#Shallow Parsing

import nltk
nltk.download('punkt')
from nltk import word_tokenize, pos_tag
from nltk.chunk import RegexpParser
text = "The small boy is playing in the garden"
tags = pos_tag(word_tokenize(text))
grammar = "NP:{<DT>?<JJ>*<NN>}"
parser = RegexpParser(grammar)
result = parser.parse(tags)
print(result)
```

```
(S
  (NP The/DT small/JJ boy/NN)
  is/VBZ
  playing/VBG
  in/IN
  (NP the/DT garden/NN))
```

4. Implementing RegExP Parser

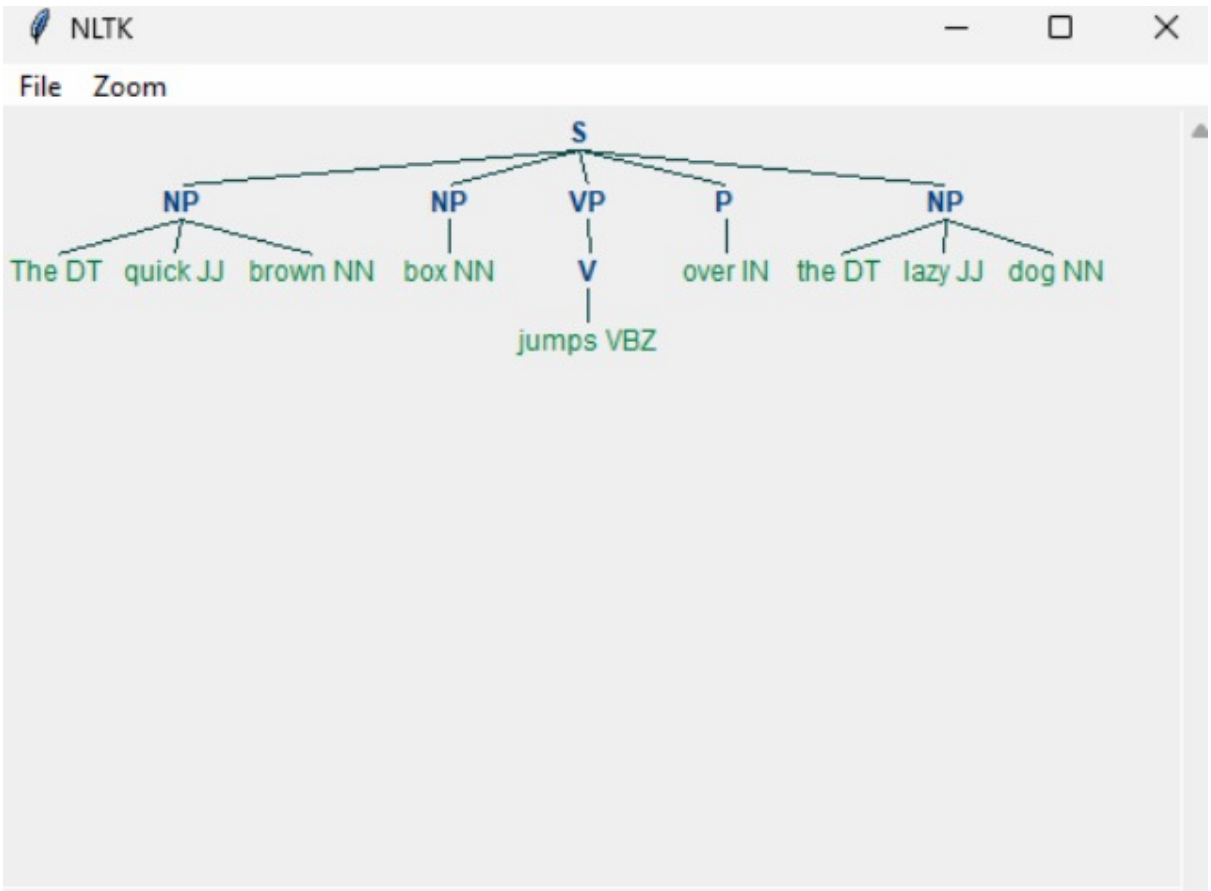
Code:

```
import nltk

from nltk import word_tokenize, pos_tag

from nltk.chunk import RegexpParser


text = 'The quick brown fox jumps over the lazy dog'

tags = pos_tag(word_tokenize(text))

reg_parser = RegexpParser("""

NP:{<DT>?<JJ>*<NN>}

P:{<IN>}

V:{<V.*>}

PP:{<IN><NP>}

VP:{<V.*><NP|PP>}

""")


result = reg_parser.parse(tags)

print("Parse Tree: ", result)

result.draw
```

Output:

```python
#Reg exp Parser

import nltk
from nltk import pos_tag, word_tokenize, RegexpParser
text = "The quick brown fox jumps over the lazy dog"
tokens = word_tokenize(text)
tags=pos_tag(tokens)
reg_parser = RegexpParser("""
NP:{<DT>?<JJ>*<NN>}
P:{<!N>}
V:{<V.*>}
PP:{<!N><NP>}
VP:{<V.*><NP|PP>*}
""")
result = reg_parser.parse(tags)
print('Parse Tree:',result)
result.draw()
```

```
Parse Tree:  (S
  (NP The/DT quick/JJ brown/NN)
  (NP fox/NN)
  (V jumps/VBZ)
  (P over/IN)
  (NP the/DT lazy/JJ dog/NN))
```



ASSIGNMENT 8

NAME: TARLANA VIKAS SLOT: L47+L48

REGD. NO.: 23BCE7267 COURSE CODE: CSE3015

COURSE NAME: NATURAL LANGUAGE PROCESSING DATE: 26-02-2026

1. Implementing Named Entity Relations.

Code:

```
#Named Entity Relations


import nltk,re
nltk.download('maxent_ne_chunker')
from nltk import word_tokenize, pos_tag, ne_chunk
from nltk.sem.relextract import extract_rels,rtuple
nltk.download('words')
text = open("/content/input.txt").read()
for sent in nltk.sent_tokenize(text):
   print("\nSentence:",sent)
   tokens = nltk.word_tokenize(sent)
   pos = nltk.pos_tag(tokens)
   tree = nltk.ne_chunk(pos)
   print("NER Tree: ",tree)
   rels =
extract_rels('PERSON','GPE',tree,corpus='ace',pattern=re.compile(r'.*'))
   for r in rels:
```

```
print("Relation: ",rtuple(r))
```

Output:

```
[nltk_data] Downloading package maxent_ne_chunker to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package maxent_ne_chunker is already up-to-date!
[nltk_data] Downloading package words to /root/nltk_data...
[nltk_data]   Package words is already up-to-date!

Sentence: Barack Obama was born in Hawaii.
NER Tree:  (S
  (PERSON Barack/NNP)
  (PERSON Obama/NNP)
  was/VBD
  born/VBN
  in/IN
  (GPE Hawaii/NNP)
  ./.)

Sentence: He was the president of the United States of America.
NER Tree:  (S
  He/PRP
  was/VBD
  the/DT
  president/NN
  of/IN
  the/DT
  (GPE United/NNP States/NNPS)
  of/IN
  (GPE America/NNP)
  ./.)
```

```
Sentence: Obama met Angela Merkel in Germany
NER Tree:  (S
  (PERSON Obama/NNP)
  met/VBD
  (PERSON Angela/NNP Merkel/NNP)
  in/IN
  (GPE Germany/NNP))
```

2. Printing Top 10 Sentences from the text file.

Code:

#Top 10 Sentences from the text file

```
import nltk,heapq

nltk.download('stopwords')

from nltk.corpus import stopwords

text = open("/content/news.txt").read()

words = [w for w in nltk.word_tokenize(text.lower())

if w.isalnum() and w not in stopwords.words('english')]

freq = {w : words.count(w) for w in words}

sent = nltk.sent_tokenize(text)

scores = {s:sum(freq.get(w,0) for w in nltk.word_tokenize(s.lower())) for s in sent}

for s in heapq.nlargest(10,scores,key=scores.get):

  print(s)
```

Output:

```
A new art exhibition opened at the city museum.
A new mobile app was launched to support local tourism.
The school introduced a new digital learning platform.
The mayor inaugurated a new public library branch.
The hospital opened a new wing for pediatric care.
The government announced new environmental protection measures.
A new science museum opened for public visits.
A new public park was opened near the river.
The hospital opened a new maternity ward.
The city council approved a new park project yesterday.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

3. Printing Last 10 Sentences from the text file.

Code:

#Last 10 Sentences from the text file

```
import nltk,heapq

nltk.download('stopwords')

from nltk.corpus import stopwords

text = open("/content/news.txt").read()

words = [w for w in nltk.word_tokenize(text.lower())

if w.isalnum() and w not in stopwords.words('english')]

freq = {w : words.count(w) for w in words}

sent = nltk.sent_tokenize(text)

scores = {s:sum(freq.get(w,0) for w in nltk.word_tokenize(s.lower())) for s in
sent}

for s in sent[-10:]:

  print(s)
```

Output:

```
The school organized a debate competition.
A new wildlife research center was established.
The art gallery held a photography exhibition.
The hospital opened a new maternity ward.
The mayor announced plans for smart city projects.
A new metro line reduced urban traffic.
The library offered free internet access to visitors.
The university launched a startup incubator program.
The community celebrated a harvest festival.
The government announced a national digital initiative.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

4. Printing First 10 + Last 10 Sentences.

Code:

```
#First 10 + Last 10 Sentences


import nltk

text = open("news.txt","r",encoding="utf-8").read()

sentences = nltk.sent_tokenize(text)

first10 = sentences[:10]

last10 = sentences[-10:]

combined = first10 + last10

print("First 10 + Last 10 Sentences: \n")

for s in combined:

  print(s)
```

Output:

```
First 10 + Last 10 Sentences:

The city council approved a new park project yesterday.
Scientists discovered a new species of butterfly in the rainforest.
The local library announced extended weekend hours.
A new bridge construction project began near the river.
Farmers reported higher crop yields this season.
The school introduced a new digital learning platform.
A community clean-up drive attracted hundreds of volunteers.
The mayor launched a campaign to reduce traffic congestion.
Researchers published findings on renewable energy efficiency.
The hospital opened a new wing for pediatric care.
The school organized a debate competition.
A new wildlife research center was established.
The art gallery held a photography exhibition.
The hospital opened a new maternity ward.
The mayor announced plans for smart city projects.
A new metro line reduced urban traffic.
The library offered free internet access to visitors.
The university launched a startup incubator program.
The community celebrated a harvest festival.
The government announced a national digital initiative.
```

5. Printing reversed sequence of the sentences.

Code:

```
#Reversed


import nltk

text = open("news.txt","r",encoding="utf-8").read()

sentences = nltk.sent_tokenize(text)

for s in reversed(sentences):

  print(s)
```

Output:

```
The government announced a national digital initiative.
The community celebrated a harvest festival.
The university launched a startup incubator program.
The library offered free internet access to visitors.
A new metro line reduced urban traffic.
The mayor announced plans for smart city projects.
The hospital opened a new maternity ward.
The art gallery held a photography exhibition.
A new wildlife research center was established.
The school organized a debate competition.
The government introduced stricter pollution controls.
A new shopping festival boosted retail sales.
The community organized a food donation drive.
The university expanded its international student intake.
The library hosted author meet-and-greet sessions.
A new IT hub attracted technology companies.
The mayor launched a campaign for clean drinking water.
The hospital added advanced surgical facilities.
The museum organized workshops for students.
A new public park was opened near the river.
The school introduced coding classes for children.
The government announced a literacy campaign.
The sports team signed promising young players.
A new industrial policy encouraged innovation.
```

6. Calculate the total number of words and sentences in the given file and the average words per a sentence.

Code:

Calculate the total number of words and sentences in the given file and the average words per a sentence.

```
import nltk

text = open("news.txt","r",encoding="utf-8").read()

sentences = nltk.sent_tokenize(text)

sentence_count = len(sentences)

words = nltk.word_tokenize(text)

word_count = len(words)

average_words_per_sentence = word_count / sentence_count

print("Total number of words:", word_count)

print("Total number of sentences:", sentence_count)

print("Average words per sentence:", average_words_per_sentence)
```

Output:

```
Total number of words: 844
Total number of sentences: 100
Average words per sentence: 8.44
```

#input.txt file

```
Barack Obama was born in Hawaii.
He was the president of the United States of America.
Obama met Angela Merkel in Germany
```

#news.txt file

```
The city council approved a new park project yesterday.

Scientists discovered a new species of butterfly in the rainforest.

The local library announced extended weekend hours.

A new bridge construction project began near the river.

Farmers reported higher crop yields this season.

The school introduced a new digital learning platform.

A community clean-up drive attracted hundreds of volunteers.

The mayor launched a campaign to reduce traffic congestion.

Researchers published findings on renewable energy efficiency.

The hospital opened a new wing for pediatric care.

Local businesses reported steady growth in sales.

A new art exhibition opened at the city museum.

The airport added more international flight routes.

Students participated in a regional science fair competition.
```

ASSIGNMENT 9

NAME: TARLANA VIKAS SLOT: L47+L48

REGD. NO.: 23BCE7267 COURSE CODE: CSE3015

COURSE NAME: NATURAL LANGUAGE PROCESSING DATE: 05-03-2026

1. Implementing Naïve Bayes Classifier

Code:

```
#Naive Bayes Classifier


from sklearn.feature_extraction.text import CountVectorizer

from sklearn.naive_bayes import MultinomialNB


#Training Data

X=["good movie","bad movie","great film","terrible film"]

y=["pos","neg","pos","neg"]


#Convert text to vectors

cv = CountVectorizer()

Xv=cv.fit_transform(X)


#Train Naive Bayes model

model = MultinomialNB().fit(Xv,y)
```

#Transform new text before prediction

test = cv.transform(["good film"])

print(model.predict(test)) #Predict

Output:

```
#Naive Bayes Classifier

from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

#Training Data
X=["good movie","bad movie","great film","terrible film"]
y=["pos","neg","pos","neg"]

#Convert text to vectors
cv = CountVectorizer()
Xv=cv.fit_transform(X)

#Train Naive Bayes model
model = MultinomialNB().fit(Xv,y)

#Transform new text before prediction
test = cv.transform(["good film"])
print(model.predict(test))  #Predict
```

```
['pos']
```

2. Implementing Support Vector Machine(SVM).

Code:

#SVM Support Vector Machine

from sklearn.feature_extraction.text import CountVectorizer

from sklearn.svm import LinearSVC

Training Data

```
X = ["good movie", "bad movie", "great film", "terrible film"]

y = ["pos", "neg", "pos", "neg"]


# Convert text to vectors

cv = CountVectorizer()

Xv = cv.fit_transform(X)


# Train SVM model

model = LinearSVC().fit(Xv, y)


# Transform new text before prediction

test = cv.transform(["good film"])

print(model.predict(test))  # Predict
```

Output:

```
#SVM Support Vector Machine
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.svm import LinearSVC

# Training Data
X = ["good movie", "bad movie", "great film", "terrible film"]
y = ["pos", "neg", "pos", "neg"]

# Convert text to vectors
cv = CountVectorizer()
Xv = cv.fit_transform(X)

# Train SVM model
model = LinearSVC().fit(Xv, y)

# Transform new text before prediction
test = cv.transform(["good film"])
print(model.predict(test))  # Predict
```

```
['pos']
```

3. Implementing Random Forest Classifier.

Code:

```
#Random Forest

from sklearn.feature_extraction.text import CountVectorizer

from sklearn.ensemble import RandomForestClassifier


# Training Data

X = ["good movie", "bad movie", "great film", "terrible film"]

y = ["pos", "neg", "pos", "neg"]


# Convert text to vectors

cv = CountVectorizer()

Xv = cv.fit_transform(X)


# Train Random Forest model

model = RandomForestClassifier().fit(Xv, y)


# Transform new text before prediction

test = cv.transform(["good film"])

print(model.predict(test))  # Predict
```

Output:

```
#Random Forest
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.ensemble import RandomForestClassifier

# Training Data
X = ["good movie", "bad movie", "great film", "terrible film"]
y = ["pos", "neg", "pos", "neg"]

# Convert text to vectors
cv = CountVectorizer()
Xv = cv.fit_transform(X)

# Train Random Forest model
model = RandomForestClassifier().fit(Xv, y)

# Transform new text before prediction
test = cv.transform(["good film"])
print(model.predict(test))  # Predict
```

```
['pos']
```

4. Implementing Text Clustering.

Code:

#Text Clustering

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.cluster import KMeans

Training Data

docs = ["I love cricket","Virat hit century","Python is fun","AI is future"]

X = TfidfVectorizer().fit_transform(docs)

labels = KMeans(n_clusters=2).fit_predict(X)

print([int(i) for i in labels])

Output:

```
#Text Clustering

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans

# Training Data
docs = ["I love cricket","Virat hit century","Python is fun","AI is future"]
X = TfidfVectorizer().fit_transform(docs)
labels = KMeans(n_clusters=2).fit_predict(X)
print([int(i) for i in labels])
```

```
[0, 1, 0, 0]
```

5. Implementing Agglomerative Clustering.

Code:

#Agglomerative Clustering

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.cluster import AgglomerativeClustering

docs = ["I love cricket","Virat hit century","Python is fun","AI is future"]

#Convert to TF-IDF vectors

X = TfidfVectorizer().fit_transform(docs)

#Convert sparse matrix to dense array

X=X.toarray()

#Apply Hierarchial Clustering

labels = AgglomerativeClustering(n_clusters=2).fit_predict(X)

print([int(i) for i in labels])

Output:

```
#Agglomerative Clustering

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import AgglomerativeClustering
docs = ["I love cricket","Virat hit century","Python is fun","AI is future"]

#Convert to TF-IDF vectors
X = TfidfVectorizer().fit_transform(docs)

#Convert sparse matrix to dense array
X=X.toarray()

#Apply Hierarchial Clustering
labels = AgglomerativeClustering(n_clusters=2).fit_predict(X)
print([int(i) for i in labels])
```

```
[0, 0, 1, 1]
```

6. Implementing Social Media Mining.

Code:

#Social Media Mining

import nltk

from nltk.sentiment import SentimentIntensityAnalyzer

nltk.download('vader_lexicon')

posts = ["I love this phone","This service is bad","AI is amazing"]

sia = SentimentIntensityAnalyzer()

for p in posts:

  score = sia.polarity_scores(p)['compound']

print(p,"->","Positive" if score>0 else "Negative")

Output:

```
#Social Media Mining

import nltk
from nltk.sentiment import SentimentIntensityAnalyzer
nltk.download('vader_lexicon')
posts = ["I love this phone","This service is bad","AI is amazing"]
sia = SentimentIntensityAnalyzer()
for p in posts:
  score = sia.polarity_scores(p)['compound']
  print(p,"->","Positive" if score>0 else "Negative")
```

```
I love this phone -> Positive
This service is bad -> Negative
AI is amazing -> Positive
[nltk_data] Downloading package vader_lexicon to /root/nltk_data...
[nltk_data]   Package vader_lexicon is already up-to-date!
```

7. Implementing Social Media Trend Detection.

Code:

#Social Media Trend Detection

#Social Mrdia Posts -- Tokenization -- Stopword Removal -- Frequency Analysis -- Trending Topics

```
import nltk

from  nltk.corpus import stopwords

nltk.download('punkt')

nltk.download('punkt_tab')

nltk.download('stopwords')
```

posts = ["AI is changing the future","Machine learning and AI are trending","I love learning AI","Deep Learning is part of AI"]

#Tokenize and remove stopwords

words = [w for p in posts for w in nltk.word_tokenize(p.lower()) if w.isalpha() and w not in stopwords.words('english')]

#Calculate frequency

freq = nltk.FreqDist(words)

#Top trending words

print("Trending Topics:", freq.most_common(5))

Output:

```
#Social Media Trend Detection

#Social Mrdia Posts -- Tokenization -- Stopword Removal -- Frequency Analysis -- Trending Topics

import nltk
from  nltk.corpus import stopwords
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
posts = ["AI is changing the future","Machine learning and AI are trending","I love learning AI","Deep Learning is part of AI"]

#Tokenize and remove stopwords
words = [w for p in posts for w in nltk.word_tokenize(p.lower()) if w.isalpha() and w not in stopwords.words('english')]

#Calculate frequency
freq = nltk.FreqDist(words)

#Top trending words
print("Trending Topics:", freq.most_common(5))
```

```
Trending Topics: [('ai', 4), ('learning', 3), ('changing', 1), ('future', 1), ('machine', 1)]
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

8. Implement a spam detection system that classifies SMS messages as spam or ham. Use a small dataset of messages, extract word features, and train a classifier to predict the category of new messages.

Code:

#Implement a spam detection system that classifies SMS messages as spam or ham.

#Use a small dataset of messages, extract word features, and train a classifier to predict the category of new messages.

```
import pandas as pd

from sklearn.feature_extraction.text import CountVectorizer

from sklearn.ensemble import RandomForestClassifier

from sklearn.model_selection import train_test_split

from sklearn.metrics import classification_report


# Load dataset from text file

# Each line: label \t message

data = pd.read_csv("sms_data.txt", sep="\t", header=None, names=["label", "message"])


# Check if data loaded correctly

print(data.head())


# Features and labels

X = data["message"].astype(str)   # ensure messages are strings

y = data["label"]


# Convert text to vectors
```

```
cv = CountVectorizer()

Xv = cv.fit_transform(X)


# Split into train/test sets

X_train, X_test, y_train, y_test = train_test_split(Xv, y, test_size=0.3,
random_state=42)


# Train Random Forest classifier

model = RandomForestClassifier(n_estimators=100, random_state=42)

model.fit(X_train, y_train)


# Predictions

y_pred = model.predict(X_test)


# Evaluation

print(classification_report(y_test, y_pred))


# Try predicting a new message

test_message = cv.transform(["Free entry in a contest, click here"])

print("Prediction:", model.predict(test_message)[0])
```

Output:

```python
#Implement a spam detection system that classifies SMS messages as spam or ham.
#Use a small dataset of messages, extract word features, and train a classifier to predict the category of new messages.

import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

# Load dataset from text file
# Each line: label \t message
data = pd.read_csv("sms_data.txt", sep="\t", header=None, names=["label", "message"])

# Check if data loaded correctly
print(data.head())

# Features and labels
X = data["message"].astype(str)   # ensure messages are strings
y = data["label"]

# Convert text to vectors
cv = CountVectorizer()
Xv = cv.fit_transform(X)

# Split into train/test sets
X_train, X_test, y_train, y_test = train_test_split(Xv, y, test_size=0.3, random_state=42)

# Train Random Forest classifier
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

# Evaluation
print(classification_report(y_test, y_pred))

# Try predicting a new message
test_message = cv.transform(["Free entry in a contest, click here"])
print("Prediction:", model.predict(test_message)[0])
```

```
                                          label  message
0          spam    Win a free iPhone now! Click here      NaN
1          ham Are we still meeting for lunch today?      NaN
2  spam    Congratulations! You have won $1000 ca...      NaN
3   ham Don't forget to bring the documents tomorrow      NaN
4      spam    Exclusive offer just for you, buy now      NaN
                                            precision    recall  f1-score   support

       ham Are we still meeting for lunch today?       0.00      0.00      0.00       1.0
           ham Can you call me when you’re free?       0.00      0.00      0.00       1.0
       spam    Win a free iPhone now! Click here       0.00      0.00      0.00       0.0
spam    You’ve been selected for a free vacation       0.00      0.00      0.00       1.0

                                        accuracy                           0.00       3.0
                                       macro avg       0.00      0.00      0.00       3.0
                                    weighted avg       0.00      0.00      0.00       3.0

Prediction: spam    Win a free iPhone now! Click here
```

ASSIGNMENT 10

NAME: TARLANA VIKAS SLOT: L47+L48

REGD. NO.: 23BCE7267 COURSE CODE: CSE3015

COURSE NAME: NATURAL LANGUAGE PROCESSING DATE: 12-03-2026

1. Implementing Web Crawler.

Code:

```
#Web Crawler

import requests

from bs4 import BeautifulSoup

#URL to crawl

url="https://www.geeksforgeeks.org/"

#Download the webpage

response = requests.get(url);

#Parse HTML content

soup = BeautifulSoup(response.content, "html.parser")

#Find all links in the page

links = soup.find_all('a')

#Print all links

for link in links:

    print(link.get('href'))
```

Output:

```
https://www.geeksforgeeks.org/
https://www.geeksforgeeks.org/dsa/dsa-tutorial-learn-data-structures-and-algorithms/
https://www.geeksforgeeks.org/explore
https://www.geeksforgeeks.org/c/c-programming-language/
https://www.geeksforgeeks.org/cpp/c-plus-plus/
https://www.geeksforgeeks.org/java/java/
https://www.geeksforgeeks.org/python/python-programming-language-tutorial/
https://www.geeksforgeeks.org/javascript/javascript-tutorial/
https://www.geeksforgeeks.org/data-science/data-science-for-beginners/
https://www.geeksforgeeks.org/machine-learning/machine-learning/
https://www.geeksforgeeks.org/courses
https://www.geeksforgeeks.org/linux-unix/linux-tutorial/
https://www.geeksforgeeks.org/devops/devops-tutorial/
https://www.geeksforgeeks.org/courses/dsa-self-paced
https://www.geeksforgeeks.org/courses/data-science-live
https://www.geeksforgeeks.org/courses/mastering-system-design-low-level-to-high-level-solutions
https://www.geeksforgeeks.org/connect/home
https://www.geeksforgeeks.org/courses
https://www.geeksforgeeks.org/courses/interviewe-101-data-structures-algorithm-system-design/
https://www.geeksforgeeks.org/courses/Java-backend-live/
https://www.geeksforgeeks.org/courses/generative-ai-training-program/
https://www.geeksforgeeks.org/courses/devops-live/
https://www.geeksforgeeks.org/courses/cpp-programming-basic-to-advanced/
https://www.geeksforgeeks.org/courses/java-online-course-complete-beginner-to-advanced/
https://www.geeksforgeeks.org/jobs
https://www.geeksforgeeks.org/gfg-hiring-solutions-for-recruiters/
https://www.geeksforgeeks.org/advertise-with-us/
https://www.geeksforgeeks.org/campus-training-program/
https://www.geeksforgeeks.org/learn-data-structures-and-algorithms-dsa-tutorial/
https://www.geeksforgeeks.org/web-development/
https://www.geeksforgeeks.org/ai-ml-ds/
https://www.geeksforgeeks.org/machine-learning/
https://www.geeksforgeeks.org/python-programming-language/
https://www.geeksforgeeks.org/java/java/
https://www.geeksforgeeks.org/system-design-tutorial/
```

2. Displaying tokens using NLTK from a webpage.

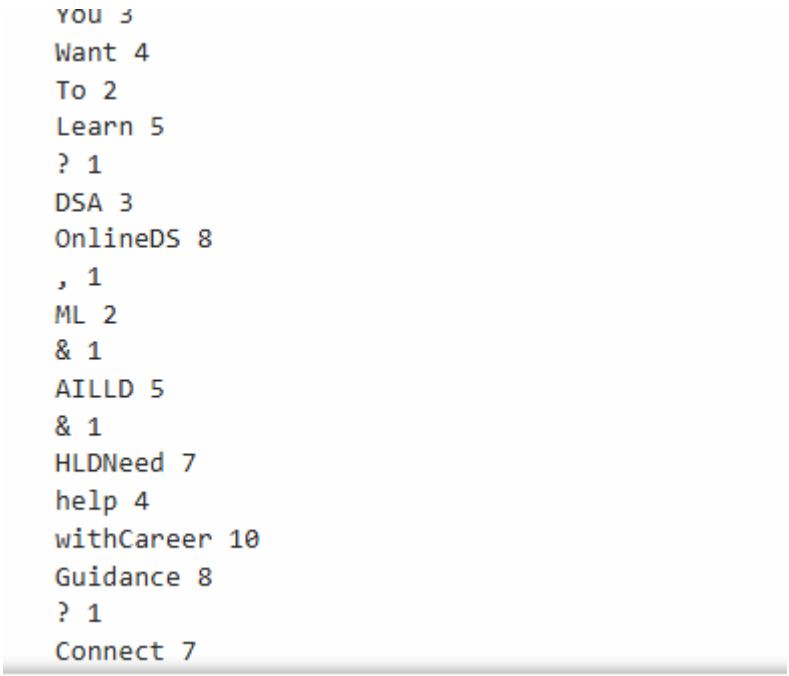
Code:

```
#Displaying tokens using NLTK from a webpage.

import requests

from bs4 import BeautifulSoup

import nltk

from nltk.tokenize import word_tokenize

#URL to crawl

url = "https://www.geeksforgeeks.org/"

#Fetch webpage

response = requests.get(url)
```

```
#Parse HTML content

soup = BeautifulSoup(response.content, "html.parser")

#Extract text from the page

text = soup.get_text()

#Tokenize text using NLTK

tokens = word_tokenize(text)

#Print first 20 tokens

print("Tokens from the webpage:")

print(tokens[:20])


for word in tokens:

    print(word, len(word))
```

Output:

```
Tokens from the webpage:
['GeeksforGeeks', '|', 'Your', 'All-in-One', 'Learning', 'PortalCoursesTutorialsPracticeJobsDSAPractice', 'ProblemsC', 'C++JavaPythonJavaScriptData'
'ScienceMachine', 'LearningCoursesLinuxDevOpsHello', ',', 'What', 'Do', 'You', 'Want', 'To', 'Learn', '?', 'DSA', 'OnlineDS']
GeeksforGeeks 13
| 1
Your 4
All-in-One 10
Learning 8
PortalCoursesTutorialsPracticeJobsDSAPractice 45
ProblemsC 9
C++JavaPythonJavaScriptData 27
ScienceMachine 14
LearningCoursesLinuxDevOpsHello 31
, 1
What 4
Do 2
You 3
Want 4
```



3. Implementing Facebook Influencer.

Code:

```
#Facebook Influencer
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
#Example Facebook posts from users
posts={
   "Alice": "I love machine learning and artificial intelligence",
   "Bob": "Machine learning is amazing and powerful",
   "Charlie": "Artificial intelligence and deep learning are the future",
   "David": "I enjoy reading books and learning new technologies",
}

stop_words=set(stopwords.words("english"))

scores={}
for user,text in posts.items():
   tokens=word_tokenize(text)
   words=[w for w in tokens if w.lower() not in stop_words and w.isalpha()]
   scores[user]=len(words)
```

#Identify influencer
influencer=max(scores,key=scores.get)

print("Influence scores:",scores)
print("Facebook influencer:",influencer)

Output:

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
#Example Facebook posts from users
posts={
    "Alice": "I love machine learning and artificial intelligence",
    "Bob": "Machine learning is amazing and powerful",
    "Charlie": "Artificial intelligence and deep learning are the future",
    "David": "I enjoy reading books and learning new technologies",
}

stop_words=set(stopwords.words("english"))

scores={}
for user,text in posts.items():
    tokens=word_tokenize(text)
    words=[w for w in tokens if w.lower() not in stop_words and w.isalpha()]
    scores[user]=len(words)

#Identify influencer
influencer=max(scores,key=scores.get)

print("Influence scores:",scores)
print("Facebook influencer:",influencer)
```

```
Influence scores: {'Alice': 5, 'Bob': 4, 'Charlie': 5, 'David': 6}
Facebook influencer: David
```

4. Implementing Text analyzer from IPL Website.

Code:
#Text analyzer from IPL Website.

import requests
from bs4 import BeautifulSoup
import nltk

```
from nltk.tokenize import word_tokenize
nltk.download('punkt_tab')
from nltk.corpus import stopwords
nltk.download('stopwords')
url = "https://www.iplt20.com/"
page = requests.get(url)
soup = BeautifulSoup(page.content, "html.parser")
text = soup.get_text()
tokens = word_tokenize(text.lower())
stop_words = set(stopwords.words('english'))
words = [w for w in tokens if w.isalpha() and w not in stop_words]
print("First 2O words:",words[:20])
print("Total words:",len(words))
```

Output:

```
#Text analyzer from IPL Website.
import requests
from bs4 import BeautifulSoup
import nltk
from nltk.tokenize import word_tokenize
nltk.download('punkt_tab')
from nltk.corpus import stopwords
nltk.download('stopwords')
url = "https://www.iplt20.com/"
page = requests.get(url)
soup = BeautifulSoup(page.content, "html.parser")
text = soup.get_text()
tokens = word_tokenize(text.lower())
stop_words = set(stopwords.words('english'))
words = [w for w in tokens if w.isalpha() and w not in stop_words]
print("First 2O words:",words[:20])
print("Total words:",len(words))
```

```
[nltk_data] Error loading punkt_tab: <urlopen error [SSL:
[nltk_data]     CERTIFICATE_VERIFY_FAILED] certificate verify failed:
[nltk_data]     unable to get local issuer certificate (_ssl.c:1006)>
[nltk_data] Error loading stopwords: <urlopen error [SSL:
[nltk_data]     CERTIFICATE_VERIFY_FAILED] certificate verify failed:
[nltk_data]     unable to get local issuer certificate (_ssl.c:1006)>
First 2O words: ['ipl', 'indian', 'premier', 'league', 'official', 'website', 'follow', 'us', 'matches', 'videos', 'latest', 'ipl', 'exclusive', 'auctio
n', 'magic', 'moments', 'highlights', 'interviews', 'press', 'conferences']
Total words: 292
```

5. Implementing Text Summarization.

Code:

```
#Text Summarization

!pip install feedparser
import feedparser,nltk
from nltk.tokenize import sent_tokenize,word_tokenize
from nltk.corpus import stopwords
from collections import Counter
```

```
feed =
feedparser.parse("https://timesofindia.indiatimes.com/rssfeedstopstories.cms
")
nltk.download('punkt')
nltk.download('stopwords')
for entry in feed.entries[:3]:
   text = entry.title + " " + entry.summary
   words = [w.lower() for w in word_tokenize(text) if w.isalpha()]
   words = [w for w in words if w not in stopwords.words('english')]
   freq=Counter(words)
   sentences = sent_tokenize(text)
   summary = sorted(sentences, key=lambda s: sum(freq[w.lower()] for w in
word_tokenize(s) if w.lower() in freq), reverse=True)[:2]
   print("\nTitle:",entry.title)
   print("\nSummary:",' '.join(summary))
```

Output:

```
Title: LS showdown continues: Speaker stops Rahul mid-speech after his remark on Hardeep Puri
Summary: LS showdown continues: Speaker stops Rahul mid-speech after his remark on Hardeep Puri Lok Sabha witnessed a heated exchange as Rahul Gandhi's speech o
Title: 'Simply not ready': US says its military cannot escort vessels in Hormuz right now
Summary: 'Simply not ready': US says its military cannot escort vessels in Hormuz right now America's military is currently unable to escort oil tankers through
Title: 'Fuel supply flowing, LPG output stepped up': Puri slams 'rumour mongering' by oppn
Summary: 'Fuel supply flowing, LPG output stepped up': Puri slams 'rumour mongering' by oppn
```

6. Program to construct a Dependency Tree

Code:

```
#Program to construct a Dependency Tree

import spacy

from spacy import displacy



#Load English NLP Model

nlp = spacy.load("en_core_web_sm")



#Parse Sentence
```

sentence = "The quick brown fox jumps over the lazy dog"

doc = nlp(sentence)

displacy.render(doc,style="dep",jupyter=True) #if using jupyter

Output: